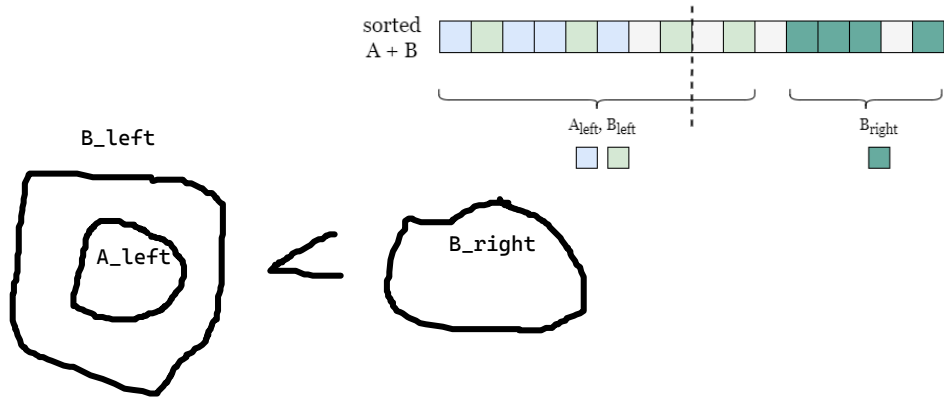
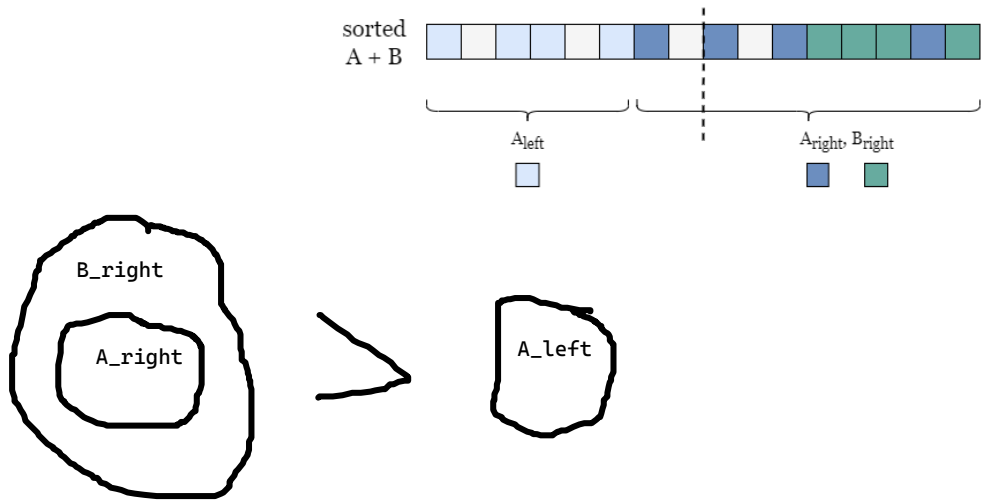


If $A_{mid} < B_{mid}$

Then we can absolutely guarantee the following:



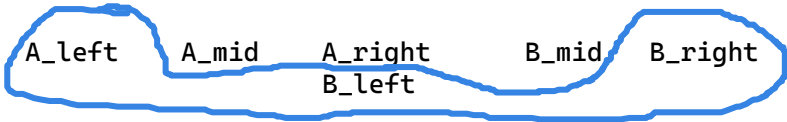
Another thing we can guarantee is:



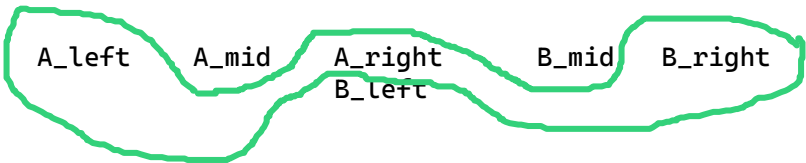
The merged array has the following layout in sorted ascending order:



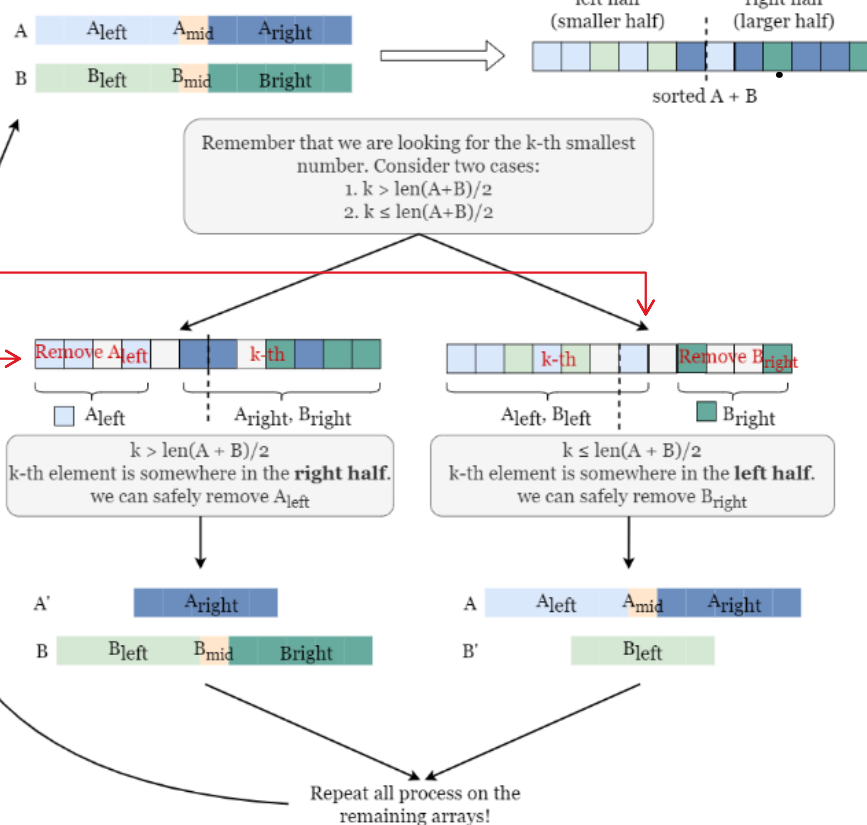
these two sets might intersect we can't guarantee the order of these



A_{left} and $B_{left} < B_{right}$



$A_{left} < A_{right}$ and B_{right}



Algorithm:

In the previous picture, we repeat all processes using the modified arrays, but this is just for the sake of understanding. We won't create copies of two arrays repeatedly, because that would introduce a linear time complexity at least. Instead, we just treat a part of the original array as the modified array for the next step, so that we can repeat the process on the original array without making any duplication. To do this, we need to maintain four pointers, two pointers for each array, e.g., `a_start` and `a_end` represent an inclusive range `[a_start, a_end]` of `A`.

Let's define a function that helps us find the k^{th} smallest element from two inclusive ranges `[a_start, a_end]` and `[b_start, b_end]` from arrays `A` and `B`.

1. If the range (for example, a range of `A`) is empty, in other words `a_start > a_end`, it means all elements in `A` are passed, we just return the `(k - a_start)`-th element from the other array `B`. Vice versa if `b_start > b_end`.
2. Otherwise, get the middle indexes of the two ranges: `a_index = (a_start + a_end) / 2`, `b_index = (b_start + b_end) / 2`.
3. Get the middle values of the two ranges: `a_value = A[a_index]`, `b_value = B[b_index]`.

ofcourse we only have 1 array remaining

Note that these are pointers

4. Cut one array in half, according to:

- If `a_index + b_index < k`, cut one smaller half.
 - If `a_value < b_value`, cut the smaller half of `A`.
 - Otherwise, cut the smaller half of `B`.
- Otherwise, cut one larger half.
 - If `b_value < a_value`, cut the larger half of `B`.
 - Otherwise, cut the larger half of `A`.

$$\frac{a_{\text{start}} + a_{\text{end}} + b_{\text{start}} + b_{\text{end}}}{2} = \frac{\text{size of } A+B}{2}$$

5. Repeat step 1 using the new starting and ending indexes of `A` and `B`.

Then we move on to find the median elements, and get the length of both arrays `na = len(A)` and `nb = len(B)`.

- If the total number of elements in `A` and `B` is odd, we just use the above function to find the middle element, that is `k = (na + nb) / 2`.
- Otherwise, we use the function to find two middle elements: `k = (na + nb) / 2 - 1` and `k = (na + nb) / 2`, and return their average.